PYTHON FOR DATA ANALYSIS

NumPy Basics • Pandas Basics • Merging, Joining & Aggregation

A beginner-friendly teaching plan with setup, guided examples, practical applications, classroom activities and a mini-project

How to use this guide

Students move from numerical arrays, to structured tabular data, and finally to combining and summarizing datasets.

Recommended teaching sequence
Concept → Simple/bookish example → Code demonstration → Practical business example → Student exercise

Learning outcomes

- Create NumPy arrays, access elements and perform numerical computations.
- Create and inspect Pandas Series and DataFrames.
- Select, filter, sort, modify and clean tabular data.
- Combine related datasets using merge(), join() and concat().
- Summarize data using groupby() and aggregation functions.
- Explain what a result means in a simple business or academic context.

At-a-glance course structure

Topic	Main focus	Suggested time
1. NumPy Basics	Arrays, indexing, slicing and computations	60–75 minutes
2. Pandas Basics	Series, DataFrames and data manipulation	75–90 minutes
3. Merging, Joining & Aggregation	Combining datasets and summarizing information	65–75 minutes
Integrated mini-project	Apply all three topics to one business scenario	30–45 minutes

1. Classroom setup

Recommended environment: Jupyter Notebook. It allows students to run code one cell at a time, inspect output immediately and experiment without executing an entire program.

Install the libraries

```
pip install numpy pandas jupyter
```

Start Jupyter Notebook

```
jupyter notebook
```

Import the libraries

```
import numpy as np
import pandas as pd
```

Explain that np and pd are conventional short aliases. They make code shorter and are widely used in Python data-analysis examples.

2. Topic 1 — NumPy Basics: Arrays & Computations

A. Introduce the problem

Ask: “Suppose I have marks for 1,000 students. Do I want to create 1,000 individual variables to store those marks?”

Explain that arrays provide an organized way to store numerical values and apply calculations to many values efficiently.

B. What is NumPy?

NumPy stands for Numerical Python. It is a Python library used for numerical computing, multidimensional arrays and mathematical operations. NumPy arrays are particularly useful for numerical data and commonly contain values of a consistent data type.

C. Bookish example — student marks

Start with a small dataset of five students. First show the values, then calculate summary statistics.

```
import numpy as np

marks = np.array([75, 80, 65, 90, 85])

print("Marks:", marks)
print("Total:", np.sum(marks))
print("Average:", np.mean(marks))
print("Highest:", np.max(marks))
print("Lowest:", np.min(marks))
```

Expected output:

```
Marks: [75 80 65 90 85]
Total: 395
Average: 79.0
Highest: 90
Lowest: 65
```

Function	Purpose
np.array()	Create an array from values.
np.sum()	Calculate the total.
np.mean()	Calculate the average.
np.max() / np.min()	Find the highest or lowest value.
np.median()	Find the middle value.
np.std()	Calculate standard deviation.

D. Array creation

```
values = np.array([10, 20, 30])
zeros = np.zeros(4)
ones = np.ones(3)
sequence = np.arange(1, 6)

print(values)
print(zeros)
print(ones)
print(sequence)
```

- np.array(): creates an array from existing values.
- np.zeros(): creates an array filled with zeros.
- np.ones(): creates an array filled with ones.
- np.arange(): generates a sequence of values.
- reshape(): changes the dimensions when the number of elements permits it.

E. Indexing and slicing

```
marks = np.array([75, 80, 65, 90, 85])

print(marks[0])    # first value: 75
print(marks[2])    # third value: 65
print(marks[1:4])  # indexes 1, 2 and 3
print(marks[-1])   # last value: 85
```

Explain that Python indexing starts at zero. In a slice, the start position is included and the stop position is excluded.

F. Array computations

```
marks = np.array([75, 80, 65, 90, 85])

print(marks + 5)
print(marks - 5)
print(marks * 2)
print(marks / 2)
```

Emphasize that NumPy applies these operations element by element. Students do not need to write a manual loop for this basic calculation.

G. Practical example — monthly sales

Scenario: A retail company wants a quick summary of its sales over six months.

```
monthly_sales = np.array([120000, 150000, 135000,
                          180000, 200000, 175000])

print("Total sales:", np.sum(monthly_sales))
print("Average sales:", np.mean(monthly_sales))
print("Best month:", np.max(monthly_sales))
print("Lowest month:", np.min(monthly_sales))

new_sales = monthly_sales * 1.10  # apply a 10% increase
print("After 10% increase:", new_sales)
```

H. Exercise

1. Create a NumPy array containing marks for 10 students.
2. Calculate the total and average marks.
3. Find the highest and lowest marks.
4. Increase every student's marks by 5.
5. Identify the marks greater than 80 using a condition.

3. Topic 2 — Pandas Basics: DataFrames & Data Manipulation

A. Introduce the problem

Ask: “Your manager gives you an Excel file containing 5,000 customer records. How would you search, filter, update and analyze that information using Python?”

Introduce Pandas as a library for working with structured, table-like data.

B. Series and DataFrame

Structure	Explanation	Familiar analogy
Series	One-dimensional labeled data structure.	One spreadsheet column.
DataFrame	Two-dimensional structure with rows and columns.	An entire worksheet or database table.

```
import pandas as pd

marks = pd.Series([75, 80, 65, 90, 85])

students = pd.DataFrame({
    "Name": ["Ali", "Ahmed", "Sara", "Fatima"],
    "Age": [20, 21, 19, 22],
    "Marks": [75, 80, 90, 85]
})

print(students)
```

Explain: each row represents one student, each column represents an attribute, each cell contains a value, and the numbers on the left are row indexes.

C. Inspecting a DataFrame

```
print(df.head())       # first five rows
print(df.tail(2))      # last two rows
print(df.shape)        # number of rows and columns
print(df.columns)      # column names
print(df.info())       # data types and non-null counts
print(df.describe())   # statistical summary
```

Command	Purpose
head() / tail()	Preview the first or last records.
shape	Return the number of rows and columns.
columns	Show column names.
info()	Show data types and non-missing counts.
describe()	Show descriptive statistics for numeric columns.

D. Selecting and filtering

```
# Select one column
print(df["Name"])
```

```
# Select multiple columns
print(df[["Name", "Marks"]])

# Keep students scoring above 80
high_scorers = df[df["Marks"] > 80]
print(high_scorers)

# Select rows and columns with loc
df.loc[df["Marks"] > 80, ["Name", "Marks"]]
```

Explain that selecting chooses columns or rows, while filtering keeps only records that meet a condition.

E. Sorting and modifying data

```
# Sort by marks, highest first
df_sorted = df.sort_values(by="Marks", ascending=False)

# Add a new column
df["Result"] = df["Marks"].apply(
    lambda x: "Pass" if x >= 50 else "Fail"
)

# Update Ali's marks
df.loc[df["Name"] == "Ali", "Marks"] = 88
```

F. Handling missing values

```
data = {
    "Name": ["Ali", "Ahmed", "Sara"],
    "Marks": [75, None, 90]
}
df = pd.DataFrame(data)

print(df.isnull().sum())

df["Marks"] = df["Marks"].fillna(0)  # replace missing value
# df.dropna()                         # remove rows with missing values
```

Explain that the correct choice depends on the meaning of the missing data. Replacing a missing mark with zero is only appropriate if that matches the business or academic rule.

G. Practical example — e-commerce orders

Scenario: An online store wants to calculate the value of each order and identify high-value purchases.

```
orders = pd.DataFrame({
    "OrderID": [101, 102, 103, 104, 105],
    "Customer": ["Ali", "Sara", "Ahmed", "Fatima", "Usman"],
    "Product": ["Laptop", "Mobile", "Mouse", "Laptop", "Keyboard"],
    "Quantity": [1, 2, 3, 1, 2],
    "Price": [150000, 50000, 2000, 150000, 3000]
})

orders["Total"] = orders["Quantity"] * orders["Price"]
high_value = orders[orders["Total"] > 10000]

print(orders.sort_values("Total", ascending=False))
print(high_value)
```

H. Exercise — employee records

Create a DataFrame with EmployeeID, Name, Department, Salary and Experience.

6. Display the first five employees.
7. Find employees earning more than Rs. 100,000.
8. Sort employees by salary.
9. Add an AnnualSalary column.
10. Calculate average salary.
11. Inspect and handle missing salary values.

4. Topic 3 — Merging, Joining & Aggregation in Pandas

A. Introduce the problem

Ask: “One file contains customer details and another contains their orders. How can we find which customer placed each order?”

Explain that real-world information is often split across multiple tables. We combine related records using a shared key or index, then summarize the combined data.

B. Understand the three terms

Operation	Simple explanation	Typical use
Merge	Match records using one or more common columns or indexes.	Match CustomerID in customers and orders.
Join	Combine DataFrames, commonly using their indexes.	Combine two tables that share an index.
Aggregation	Summarize records using calculations.	Total sales, average marks or order counts.

C. Bookish example — student details and results

```
students = pd.DataFrame({
    "StudentID": [1, 2, 3, 4],
    "Name": ["Ali", "Sara", "Ahmed", "Fatima"],
    "Class": ["A", "B", "A", "B"]
})

results = pd.DataFrame({
    "StudentID": [1, 2, 3, 4],
    "Marks": [75, 90, 65, 85]
})

combined = pd.merge(students, results, on="StudentID")
print(combined)
```

Explain that StudentID is the common key. Pandas matches records with the same ID and returns a combined DataFrame.

D. Four common merge types

Merge type	What it returns
Inner	Only keys that have a match in both datasets.
Left	All rows from the left dataset, plus matching rows from the right.
Right	All rows from the right dataset, plus matching rows from the left.
Outer	All keys from both datasets; unmatched fields are missing.

```
pd.merge(left, right, on="ID", how="inner")
pd.merge(left, right, on="ID", how="left")
pd.merge(left, right, on="ID", how="right")
pd.merge(left, right, on="ID", how="outer")
```

For a useful classroom demonstration, add an ID that exists in only one of the two tables. Students can then observe how the four merge types differ.

E. Joining and concatenating

```
# join() commonly combines DataFrames using their index
combined = students.set_index("StudentID").join(
    results.set_index("StudentID")
)

# concat() stacks DataFrames along rows by default
all_students = pd.concat([class_a, class_b], ignore_index=True)
```

A simple distinction: merge matches on keys; join is commonly index-based; concat appends or stacks along an axis.

F. Aggregation using groupby()

Ask: “A school has students in different classes. How can we calculate the average marks for each class?”

```
students = pd.DataFrame({
    "Name": ["Ali", "Sara", "Ahmed", "Fatima", "Usman"],
    "Class": ["A", "B", "A", "B", "A"],
    "Marks": [75, 90, 65, 85, 80]
})

average_by_class = students.groupby("Class")["Marks"].mean()
print(average_by_class)
```

Break down the expression: groupby("Class") creates groups, ["Marks"] selects the values to summarize, and mean() calculates the average for each group.

Function	Purpose
sum()	Total value.
mean()	Average value.
count()	Count of non-missing values.
min() / max()	Smallest or largest value.
median()	Middle value.

```
students.groupby("Class").agg({
    "Marks": ["mean", "max", "min", "count"]
})
```

G. Practical example — customer sales analysis

Scenario: An e-commerce company stores customer details separately from order details. Management wants to know how much each customer spends and which city generates the most revenue.

```
customers = pd.DataFrame({
    "CustomerID": [1, 2, 3, 4],
    "Name": ["Ali", "Sara", "Ahmed", "Fatima"],
    "City": ["Lahore", "Karachi", "Lahore", "Islamabad"]
})

orders = pd.DataFrame({
    "OrderID": [101, 102, 103, 104, 105, 106],
    "CustomerID": [1, 2, 1, 3, 4, 2],
    "Amount": [5000, 8000, 3000, 6000, 9000, 2000]
})

# Step 1: combine customer and order information
sales = pd.merge(customers, orders, on="CustomerID", how="inner")
```

```
# Step 2: total spending per customer
customer_totals = sales.groupby(["CustomerID", "Name"])["Amount"].sum()

# Step 3: revenue by city
city_totals = sales.groupby("City")["Amount"].sum()

# Step 4: multiple metrics by city
city_report = sales.groupby("City").agg({
    "Amount": ["sum", "mean", "count"]
})

print(customer_totals)
print(city_totals)
print(city_report)
```

H. Exercise — employee attendance

Provide two DataFrames: an employee table (EmployeeID, Name, Department) and an attendance table (EmployeeID, AttendanceDays, WorkingDays). Ask students to:

12. Merge the two datasets using EmployeeID.
13. Calculate attendance percentage for each employee.
14. Calculate average attendance by department.
15. Identify employees with attendance below 80%.
16. Find the department with the highest average attendance.

5. Integrated mini-project — Retail Sales Analysis

Business scenario

You are a junior data analyst at a retail company. The company provides three files: Customers.csv, Orders.csv and Products.csv. Management wants a concise analysis of sales performance.

Suggested tasks

Part 1 — NumPy

- Create an array of monthly sales.
- Calculate total, average, highest and lowest sales.
- Apply a 10% increase to the sales array.

Part 2 — Pandas

- Load the CSV files using pd.read_csv().
- Inspect the datasets using head(), info(), shape and describe().
- Filter and sort orders by amount.
- Check for missing values and handle them appropriately.

Part 3 — Merging and aggregation

- Merge customer information with orders using CustomerID.
- Merge order information with product details using ProductID.
- Calculate total sales by product.
- Calculate revenue by city.
- Identify the top five customers by spending.
- Create a summary report with totals, averages and counts.

Expected deliverable

A Jupyter Notebook containing the code, outputs and a short written explanation of the business insights. Students should be able to explain what each result means.

Suggested assessment rubric

Assessment area	What to look for	Weight
Correct setup and data loading	Libraries imported; files loaded correctly.	10%
NumPy operations	Arrays and calculations are correct.	15%
Pandas manipulation	Selection, filtering, sorting and cleaning work.	25%
Merging and aggregation	Correct keys, joins and summary calculations.	30%
Interpretation and communication	Clear explanation of findings and limitations.	20%

6. Instructor delivery plan

Suggested schedule

Session	Coverage	Time
1	Introduction and environment setup	15 min
2	NumPy arrays, creation and dimensions	30 min
3	NumPy indexing, computations and exercise	30–45 min
4	Pandas Series, DataFrames and inspection	30 min
5	Filtering, sorting, modification and missing values	40–50 min
6	Pandas practical exercise	20 min
7	Merging and joining	35 min
8	Groupby and aggregation	30 min
9	Integrated business case and review	30–45 min

Estimated total: approximately 4 hours 30 minutes to 5 hours, excluding breaks. Adjust the pace based on students' prior Python experience.

Recommended classroom rhythm

Step	What the instructor does
1. Ask	Start with a relatable problem or question.
2. Explain	Introduce one concept using plain language.
3. Demonstrate	Work through a small, bookish example.
4. Apply	Change the scenario to a practical business use case.
5. Practice	Let students write, predict and debug code.

Suggested time balance
Spend roughly 20% on concepts, 30% on guided examples and 50% on hands-on practice. The exact balance can vary by class.

Use Excel as a bridge to Pandas

Familiar Excel concept	Pandas equivalent
Worksheet / table	DataFrame
Column	Series
Filter	Boolean filtering
Sort	sort_values()
VLOOKUP-style matching	merge()
Pivot Table-style summary	groupby() / pivot_table()
SUM / AVERAGE	sum() / mean()

Core functions to prioritize

NumPy	Pandas	Combining & analysis
array()	Series(), DataFrame()	merge()
arange(), zeros(), ones()	read_csv()	join()
reshape()	head(), tail(), info()	concat()
Indexing and slicing	loc, iloc, sort_values()	groupby()
sum(), mean(), max(), min()	isnull(), fillna(), dropna()	agg(), sum(), mean(), count()